

Network and Operating Systems

Based on pre-workshop discussion from
Aditya Akella, Radhika Mittal, Keith Winstein, Minlan Yu

What are the right metrics of success?

- Provider-centric metrics:
 - SLA for throughput & latency
 - vCPU oversubscription rate, VM memory allocation
- App-centric metrics:
 - Per-job completion time relative to deadline/importance?
 - Per-application QoE?
- “Long-term” metrics:
 - Barrier to innovation / entry in cloud computing?
 - Healthy competitive ecosystem of cloud providers after a decade?
- All of these are available at different timescales.

—

What are the right abstractions between app and provider?

- low-level primitives: VMs, containers, functions?
- high-level services: database, collective compute (MapReduce), ML training/inference?
- compute vs. I/O (network and storage): is there a prudent separation?
- expose accelerators and specialized hardware? abstract over them?
- how to convey “figure-of-merit” metrics to infrastructure?
- to make a resource allocation/scheduling decision, infrastructure needs to be able to predict the consequences upfront. How?

What's the right economic model?

- Payment for effort (tenancy duration x CPU speed x RAM allocation x NIC speed)
- “Payment for results” of a high-level abstraction (quoted cost per MapReduce)
- “Payment for results” of an arbitrary computation

Cloud-to-user interface

- Challenges of IaaS/FaaS:
 - Computing infrastructure increasingly consolidated to handful of “hyperscalers”
 - Typical end-to-end efficiency understood to be abysmal, but
 - Apps do their own I/O mixed with computation: hard for provider to help
 - Billing model misaligns incentives (“paying for effort”)
 - Time constants discourage “packet-switched” computing ($\sim 10^{-1} - 10^{+1}$ seconds)
- Opportunities:
 - Getting the abstraction right (compute-I/O separation? dataflow visibility?)
 - Getting the economic model right (“paying for results”)
 - UHD multitenancy, heterogeneity, massive burst parallelism, IFC security
 - *Or: higher-level “stickier” cloud services?*

Cloud

- Heterogeneous set of accelerators and links inside and across servers
 - Hard to establish communication across different accelerators
 - Hard to establish communication across many hops of different links
- Applications
 - Microservices: Network latency becomes comparable to computing
 - Memory gets further away from compute
 - Many sources of noises that increases variations
 - Opportunity: Migration becomes easier
- Key question:
 - What's the right networking abstraction for cloud applications?
 - For performance, reliability, evolvability, manageability etc.

Key OS Questions

- Where to put the functions?
 - User space, kernel space, hardware
- Connection with other research areas
 - Cross-layers: Computer architectures, operating systems, distributed systems
 - Cross-components: Storage, computing, network

Network Design for Next-Gen Applications

- Applications such as IoT, AR/VR, robotics have stringent throughput and latency requirements.
- How do we re-think networking protocols and mechanisms to meet these requirements?
 - May not need complete perfect reliability, but some notion of partial reliability.
 - Can queue management schemes be customized to drop semantically less useful packets?
 - How would a private network customized to these applications (e.g. in a digital farm, smart factory, etc) differ from a generic network?
 - What parameters and abstractions should the network expose to enable such customizability?

Data Driven Networking 2.0

- Opportunities
 - Better systems collect rich and fine-grained data across layers and components
 - LLMs and AI become evermore powerful
 - DDN 1.0: use data available *today* to *add* to network functionality
- 2.0: Data as the narrow waist
 - Extract richer richer features with better monitoring systems
 - Data driven algorithms and design -- data to collect vs algorithm design
 - Impact on protocol stacks and implementations

DDN 2.0

- Benefits
 - Rich algorithmic and implementation playground, touching algorithms, software, hardware, and architecture.
 - Can spur fundamental networking and systems innovations
 - Informed instead of adhoc decisions, adaptativity, composability
 - Free developers from incremental heuristics development, focus on architectural questions, let AI/ML take care of fine-tuning
 - Using LLM to convert RFCs to protocol implementations

Scaffolding for DDN on public Internet

- Background:
 - Researchers have created many ML network algorithms
 - for CC, ABR, TE, CDN caching, network analytics, nowcasting, ...
 - Few of these schemes have survived deployment on public Internet
- Internet is a particularly challenging setting for ML
 - decentralized & partially observable (less observable than self-driving car)
 - can't be simulated or emulated well today (“sim2real”)
 - changes over time, sometimes adversarially
 - has rare events that matter (heavy-tailed)
- Opportunity: training wheels for crossing the chasm, e.g.:
 - Every x, research community runs internal bakeoff and nominates 3 algs
 - Public site(s) test on 0.0001% of real traffic & reports results
 - Virtuous feedback cycle helps calibrate research environments and reality